Web Engineering

Semester Project

BSSE- 6th Grey Semester

Student Details

Sr. #	Name	Roll No.
1	**Muttahir Ali**	**23010101-082**
2	**Abdullah Waryah**	**23010101-093**

Course Instructor

Prof Museb Khalid

Faculty of Computing and IT

Department of Software Engineering

Express.js & MongoDB Integration

1. Introduction

For this deliverable I built the backend for my GPA/CGPA Calculator project using Express.js and connected it to a MongoDB database through Mongoose. The frontend was already built in Vue 3, so the goal here was to give it a real place to save and retrieve GPA records instead of everything living only in the browser. This report explains how the backend is structured, how the database schema was designed, and how the two sides (Vue frontend and Express backend) talk to each other.

2. Project Structure

I organized the backend into a /server folder that sits alongside the existing /client folder from Deliverable 1, so the whole project stays in one repository as required. Inside /server, the code is split by responsibility instead of dumping everything into one file:

```
server/
├── server.js          (entry point, connects DB, starts app)
├── models/
│   └── Student.js     (Mongoose schema for a GPA record)
├── routes/
│   └── studentRoutes.js (all /api/students endpoints)
└── .env               (local secrets — MONGO_URI, PORT)
```

I kept the model and the routes in separate files because that's the standard MVC-style split for Express projects, and it makes it much easier to find and edit one piece without scrolling through everything else.

3. Database Schema Design

The database only needs one collection for now, called Student, since the app's whole job is to store the numbers a user enters into the calculator. I designed the schema to match exactly what the Vue calculator already computes, so no extra transformation is needed between frontend and database:

Field	Type	Required	What it stores
semesterSGPA	Number	Yes	The GPA calculated for one semester.
overallCGPA	Number	Yes	The cumulative GPA across all semesters up to that point.
semesterHours	Number	Yes	Credit hours taken in that semester.
totalHours	Number	Yes	Total credit hours completed so far.

I made all four number fields required because a record with a missing GPA or hours value wouldn't be meaningful to display later. Each field is stored as a Number so calculations and comparisons can be done directly on the data without extra conversion.

4. API Routes (Express)

All routes are mounted under /api/students in server.js, and the actual route logic lives in routes/studentRoutes.js. For this deliverable I implemented the two core operations the calculator needs: saving a new record and retrieving all saved records.

Method	Endpoint	Purpose
POST	/api/students	Creates a new SGPA/CGPA record in the database.
GET	/api/students	Returns all saved records (used to show history).

Both routes follow the same pattern: try to perform the Mongoose operation, send back a success response with the data if it works, and catch any errors in a try/catch block to send back a clear JSON error message instead of letting the server crash. The POST route creates a new Student document from req.body and saves it, returning the saved record with a 201 status. The GET route uses Student.find() with no filter to pull back every record in the collection, which the frontend uses to display a history of past calculations.

5. Connecting to MongoDB

The database connection is handled in server.js using Mongoose's connect() method, wrapped in a try/catch inside an async function so that connection errors get logged clearly instead of silently failing. The actual connection string is stored in a MONGO_URI variable inside a .env file, which is loaded using the dotenv package. This keeps the database credentials out of the source code.

The .env file itself is kept out of version control, since it holds the real database credentials and should never be pushed to a shared GitHub repository. Anyone else running this project locally would create their own .env file with their own MONGO_URI value.

6. Middleware

Two middleware functions are applied globally in server.js before the routes:

- **cors()** — allows the Vue frontend (running on a different port during development) to make requests to this Express server without being blocked by the browser's same-origin policy.
- **express.json()** — parses incoming JSON request bodies so that req.body works correctly inside the route handlers, for example when the frontend sends a new SGPA/CGPA record to save.

7. Testing the API

Before connecting the Vue frontend, I tested each endpoint directly using Postman/Thunder Client style requests against http://localhost:5000/api/students: sending a POST with sample SGPA/CGPA values to confirm it saves correctly, a GET to confirm the saved record comes back, which was useful for a quick check that the server was running at all before testing the real endpoints.

8. Reflection

The biggest thing I learned in this deliverable was how the frontend and backend actually stay separate but connected the Vue app doesn't know or care that MongoDB exists, it just calls a URL and gets JSON back. Splitting the model, routes, and server setup into different files also made the code much easier to reason about compared to writing everything in one script. This structure gives a solid foundation to build on for the next deliverable.